

Security Tool Selection Rationale

Big Chemistry — XDR Architecture

2026-03-06

Status: Approved but not complete - SDN missing

Date: 2026-03-06

Scope: Comprehensive comparison and justification for every security tool in the XDR architecture

Constraint: All tooling must be fully open-source and portable to on-premises bare metal

Contents

1	Executive Summary	2
1.1	Selection Criteria (Applied to All Categories)	5
1.2	Complete Tool Selection Summary	5
2	Network Intrusion Detection/Prevention System (NIDS/IPS)	5
2.1	Candidates Evaluated	5
2.2	Selection Rationale	6
3	Network Security Monitoring (NSM)	7
3.1	Candidates Evaluated	7
3.2	Selection Rationale	7
4	XDR / SIEM Platform	10
4.1	Candidates Evaluated	10
4.2	Selection Rationale	11
5	Container Runtime Security (Syscall Monitoring)	12
5.1	Candidates Evaluated	12
5.2	Selection Rationale	12
6	eBPF Runtime Security (Process/File/Network Enforcement)	13
6.1	Candidates Evaluated	13
6.2	Selection Rationale	13
7	CNI / Software-Defined Networking	15
7.1	Candidates Evaluated	15
7.2	Selection Rationale	15
8	Kubernetes Admission Control	17
8.1	Candidates Evaluated	17
8.2	Selection Rationale	17

9 Threat Intelligence Platform	18
9.1 Candidates Evaluated	18
9.2 Selection Rationale	18
10 Log Shipping / Collection	19
10.1 Candidates Evaluated	19
10.2 Selection Rationale	20
11 Monitoring Stack (Metrics + Visualisation + Logs)	20
11.1 Candidates Evaluated	20
11.2 Selection Rationale	21
12 Web Application Firewall (WAF)	22
12.1 Candidates Evaluated	22
12.2 Selection Rationale	22
13 NAT Gateway (Software-Based)	23
13.1 Candidates Evaluated	23
13.2 Selection Rationale	23
14 Licensing Summary	24
14.1 Disqualified Alternatives (Licensing)	25
15 Architecture Integration Map	25
15.1 Current Deployment: Monitoring-Only XDR Overlay	25
15.2 Recommended Target: Full Security Architecture	26
15.3 Key Differences Between Current and Target	26

1 Executive Summary

This document provides a rigorous, tool-by-tool comparison and selection rationale for every security component in the Big Chemistry 2-VPC XDR security architecture. Each tool was evaluated against 2–5 competing alternatives across standardised criteria: core capabilities, performance, Kubernetes integration, bare-metal portability, licensing, community maturity, and integration with the broader security stack.

The architecture comprises **12 tool categories** spanning network, host, container, and cloud security layers. Every selected tool is fully open-source (OSI-approved licence), deployable on bare metal without cloud dependencies, and actively maintained by a substantial community.

Table 1: Selection criteria applied to all tool categories

#	Criterion	Weight	Description
1	Fully Open-Source	Critical	OSI-approved licence, no paid tiers required for core security functionality
2	Bare-Metal Portable	Critical	Runs on-premises without cloud service dependencies
3	Kubernetes-Native	High	DaemonSet/StatefulSet deployment, Helm chart, CRD support where applicable
4	Active Community	High	Regular releases, responsive maintainers, CNCF backing preferred
5	Stack Integration	High	Works with Wazuh XDR, Prometheus metrics, Grafana dashboards, Fluent Bit shipping
6	Performance	Medium	Low overhead, efficient resource usage
7	ARM64 Support	Low	Compatible with Graviton/ARM64 node groups

Table 2: Complete tool selection summary

Category	Selected Tool	Licence	Alternatives Evaluated
Network IDS/IPS	Suricata 7.x	GPLv2	Snort 3, Zeek (IDS mode), OSSEC
Network Security Monitoring	Zeek 7.x	BSD	Arkime, ntopng, Suricata (NSM mode)
XDR / SIEM Platform	Wazuh 4.x	GPLv2 + Apache 2.0	Elastic Security, OSSIM, The-Hive+Cortex, Security Onion
Container Runtime IDS	Falco	Apache 2.0	Tracee, KubeArmor, Sysdig OSS
eBPF Runtime Security	Tetragon	Apache 2.0	bpfftrace, Tracee, Inspektor Gadget
CNI / SDN	Cilium + Hubble	Apache 2.0	Calico, Weave Net, Flannel, Antrea
Admission Control	Kyverno	Apache 2.0	OPA/Gatekeeper, Kubewarden, jsPolicy
Threat Intelligence	MISP	AGPLv3	OpenCTI, CRITs, Yeti
Log Shipping	Fluent Bit	Apache 2.0	Fluentd, Vector, Logstash, Filebeat
Monitoring Stack	Prometheus + Grafana + Loki	Apache 2.0 / AGPLv3	ELK, Victoria-Metrics, Graylog
Web Application Firewall	ModSecurity / Coraza	Apache 2.0	NAXSI, lua-resty-waf
NAT Gateway	iptables/nftables NAT VM	GPLv2	AWS NAT Gateway, pfSense, VyOS

1.1 Selection Criteria (Applied to All Categories)

1.2 Complete Tool Selection Summary

2 Network Intrusion Detection/Prevention System (NIDS/IPS)

Selected: Suricata 7.x

2.1 Candidates Evaluated

Criterion	Suricata 7.x	Snort 3	Zeek (IDS mode)	OSSEC (Network)
Primary Function	Multi-threaded NIDS/IPS	Single/multi-threaded NIDS/IPS	Passive protocol analysis	Host-based IDS
Inline IPS Mode	Yes (AF_PACKET, Netmap, DPDK)	Yes (DAQ framework)	No (passive only)	No (host-level only)
Threading	Native multi-threading (scales linearly with cores)	Multi-threaded (Snort 3, improved from Snort 2)	Single-threaded per worker (cluster mode for parallelism)	Single-threaded
Protocol Analysis	Deep (HTTP, TLS, DNS, SMB, SSH, FTP, SMTP, QUIC, HTTP/2)	Deep (HTTP, DNS, SMB, SSH)	Deepest (60+ protocol analysers, Spicy framework)	N/A
EVE JSON Logging	Native (structured JSON for all events)	Unified2 + JSON alert output	TSV/JSON logs per protocol	Syslog/JSON alerts
Rule Ecosystem	ET Open (~45,000 rules), ET Pro, custom Lua	Snort Community (~3,500), Snort Subscriber (~30,000), Talos	Zeek scripts (behavioural, not signature)	OSSEC rules (host-focused)
Throughput	10+ Gbps (multi-core, AF_PACKET)	1–5 Gbps (improved in Snort 3)	1–10 Gbps (cluster mode)	N/A (not network throughput)
Lua Scripting	Yes (custom detection logic)	Yes (Snort 3 Lua plugins)	Zeek scripting language (more powerful)	No
MISP/TI Integration	ET rules + MISP via Suricata-Update	Snort Subscriber rules	Intel framework (native MISP support)	CDB lists
MITRE ATT&CK Coverage	~200+ techniques via ET Open	~150+ via Snort Community/Subscriber	Behavioural (technique-specific scripts)	~100 (host-focused)
DPDK/AF_PACKET/Netmap support	Yes, fully supported	DAQ framework (AF_PACKET, DPDK via DAQ modules)	AF_PACKET, PF_RING	N/A
Licence	GPLv2	GPLv2	BSD	GPLv2
Community	Very active (OISF foundation, ~5K GitHub stars)	Active (Cisco/Talos, ~2K GitHub stars)	Very active (Corelight backing, ~10K GitHub stars)	Superseded by Wazuh
Bare-Metal	Excellent (native packages, all Linux)	Good (native packages)	Excellent (native, cluster mode)	Good

2.2 Selection Rationale

Suricata was selected for the inline IPS role based on four decisive requirements:

1. **Inline IPS capability.** Suricata operates in true inline mode via AF_PACKET IPS, intercepting and blocking traffic in real-time. Zeek is passive-only and cannot block traffic. This is non-negotiable for the dual Suricata IPS sandwich architecture (ingress + egress inspection).
2. **Native multi-threading.** Suricata distributes packet processing across all available CPU cores automatically, achieving 10+ Gbps throughput on modern hardware. Snort 3 improved its threading model significantly from Snort 2 but Suricata's multi-threaded architecture is more mature and better tested at high throughput.
3. **EVE JSON structured output.** Suricata produces structured JSON logs (EVE format) covering alerts, HTTP, DNS, TLS, flow, file transactions, and protocol metadata. This output integrates directly with Wazuh's native Suricata module for real-time correlation — no custom parsing required. Snort's native output formats require additional transformation.
4. **Rule ecosystem breadth.** The ET Open ruleset provides ~45,000 signatures with daily updates, covering network threats, malware C2, exploit kits, and protocol anomalies. Combined with MISP threat intelligence feeds via `suricata-update`, this provides comprehensive signature-based coverage. Snort's free Community ruleset is significantly smaller (~3,500 rules).

Why not Zeek? Zeek is selected separately as the NSM tool (Section 3). Zeek and Suricata are complementary, not competing: Suricata provides signature-based detection and inline blocking; Zeek provides deep protocol analysis and behavioural scripting. The architecture deploys both.

Why not Snort 3? Snort 3 is a capable IPS but Suricata offers superior multi-threading performance, a larger free rule ecosystem (ET Open vs Snort Community), and more mature EVE JSON output for SIEM integration. Snort's strongest ruleset (Snort Subscriber/Talos) requires a paid subscription.

3 Network Security Monitoring (NSM)

Selected: Zeek 7.x

3.1 Candidates Evaluated

Criterion	Zeek 7.x	Arkime	ntopng	Suricata (NSM mode)
Primary Function	Protocol analysis & behavioural detection	Full packet capture & indexed search	Flow analysis & traffic monitoring	Signature detection + protocol logging
Protocol Depth	Deepest (60+ analysers, Spicy parser framework)	Session indexing (application identification)	nDPI application identification	Deep (HTTP, TLS, DNS, SMB, etc.)
Output Format	Structured logs (TSV/JSON per protocol)	Indexed PCAP + session database	Dashboard + NetFlow/sFlow	EVE JSON
Full PCAP	No (metadata only)	Yes (primary function)	No	Optional
Behavioural Detection	Yes (Zeek scripting language — Turing-complete)	Retrospective search only	Flow anomaly detection	Signature-based only
Scripting	Zeek language (domain-specific, very powerful)	WISE plugins	Limited	Lua (limited scope)
JA3/JA4 Fingerprinting	Native	Via Zeek/Suricata integration	Via nDPI	JA3 native, JA4 via plugin
Intel Framework	Native (MISP, STIX, custom feeds)	WISE enrichment plugin	Basic	Via suricata-update
Cluster Mode	Yes (manager/work-er/proxy architecture)	Yes (multi-node capture)	Single instance	AF_PACKET cluster mode
K8s Deployment	DaemonSet (community patterns)	StatefulSet (storage-heavy)	Container (limited)	DaemonSet
Resource Requirements	High (CPU-intensive protocol analysis)	Very high (storage for PCAP)	Low–medium	Medium–high
Licence	BSD	Apache 2.0	GPLv3 / Commercial	GPLv2
Community	Very mature (since 1995, Corelight backing, ~10K stars)	Mature (since 2012, ~2K stars)	Mature (since 2013, ntop srl)	See Section 2

3.2 Selection Rationale

Zeek was selected for **passive network metadata analysis** for three reasons:

1. **Unmatched protocol analysis depth.** Zeek’s 60+ protocol analysers and the Spicy parser framework provide the deepest protocol visibility of any open-source tool. It generates structured logs for HTTP transactions, DNS queries, TLS certificates, SMB file transfers, SSH sessions, and dozens more — all with rich metadata that Suricata’s EVE JSON does not capture at the same level of detail.
2. **Behavioural detection via scripting.** Zeek’s scripting language enables custom behavioural detections beyond signature matching: detecting DNS tunnelling patterns, bea-

coning analysis, unusual JA3 fingerprints, certificate anomalies, and lateral movement indicators. These complement Suricata's signature-based approach.

3. **Threat intelligence integration.** Zeek's Intel framework natively consumes IOCs from MISP and produces alerts when observed in network traffic, creating a direct MISP → Zeek → Wazuh detection pipeline.

Why not Arkime? Arkime provides full packet capture, which is valuable for forensic investigation but demands massive storage infrastructure (terabytes per day at scale). The architecture preserves forensic capability through Suricata's optional PCAP capture and Zeek's metadata logs, which provide sufficient forensic context without Arkime's storage overhead. Arkime could be added as a future enhancement if full PCAP retrieval becomes a requirement.

Why not ntopng? ntopng excels at real-time traffic flow visualisation but provides less protocol analysis depth than Zeek and its Enterprise features (SNMP monitoring, advanced alerting) require a commercial licence. Its GPLv3 licence is also more restrictive than Zeek's BSD licence.

4 XDR / SIEM Platform

Selected: Wazuh 4.x (with OpenSearch Indexer)

4.1 Candidates Evaluated

Criterion	Wazuh 4.x	Elastic Security	OSSIM	TheHive + Cortex	Security Onion
Primary Function	Unified XDR/SIEM	SIEM + EDR + XDR	SIEM with bundled tools	Incident Response (SIRP)	NSM + SIEM distribution
Log Collection	Agent + agentless + cloud modules	300+ integrations (Elastic Agent)	Syslog + plugins (200)	None (receives alerts via API)	Suricata + Zeek + Elastic Agent
Event Correlation	Hierarchical rule engine (4,500+ rules)	EQL + ML anomaly detection (1,100+ rules)	Correlation directives (limited)	None (case management only)	Sigma rules + Elastalert
Active Response	Native framework (<10s execution)	Requires Platinum licence	OSSEC legacy (limited)	Cortex responders (case-driven)	Manual + Playbook
FIM	Real-time (inotify-based)	Polling-based (Auditbeat)	Outdated OSSEC 2.x	None	Via Elastic Agent (basic)
Vulnerability Detection	Native (NVD CVE scanning, agent-based)	Third-party only	OpenVAS embedded	None	None
Compliance	Native SCA module (CIS, PCI DSS, HIPAA, GDPR, NIST)	Paid tier only (CSPM/KSPM)	Basic PCI DSS	None	None
Rootkit Detection	Native	No	OSSEC legacy	None	None
Container / K8s	DaemonSet agent, K8s audit decoder	KSPM (licence tier), K8s integration	None	None	osquery (limited)
AWS Cloud Integration	Native (CloudTrail, GuardDuty, VPC Flow Logs, S3)	Native (100+ AWS sources)	None	None	None
K8s Deployment	Official Helm manifests (StatefulSet)	ECK Operator (mature)	No K8s support	Community Helm (possible)	Not supported (Salt/Docker)
Clustering / HA	Manager cluster (3-node) + Indexer cluster (3-node)	Elasticsearch (industry-leading)	Single server only	Cassandra backend	Distributed (manager + sensor + search)
Agent Footprint	~50–100 MB RAM, <2% CPU	~200–500 MB RAM, 2–5% CPU	No native agent	No agents	No proprietary agent
API	Full REST API (agent, rules, alerts, response)	Extremely comprehensive	Limited	Full REST API (well-designed)	Limited (SOC API)
MITRE ATT&CK	~250+ techniques mapped	~300+ techniques mapped	~100 (network only)	N/A	~200+ (network via Suricata/ZeeK)
Licence	GPLv2 + Apache 2.0 (no feature gating)	Elastic Licence v2 (NOT OSI-approved)	GPLv2 (stagnant)	AGPL (Community limited)	GPLv2
Feature Gating	None — all features free	Critical: ML, response, CSPM require Platinum (\$\$\$)	None (but unmaintained)	TheHive 5.x Community limits multi-tenancy	None
Community	~11K stars, very active, quarterly releases	~70K stars (all Elastic), massive	~1K stars, stagnant	~10K stars, moderate	~3K stars, active
Bare-Metal	Excellent (native packages, no cloud deps)	Good (but ELv2 licence caveat)	ISO-based only	Good (Docker/-packages)	Excellent (ISO, purpose-built for on-prem)

4.2 Selection Rationale

Wazuh is the only platform that satisfies all six critical requirements simultaneously:

1. **Licensing transparency.** Wazuh is the only platform providing a complete XDR/SIEM under a genuinely open-source licence (GPLv2) with zero feature gating. Elastic Security shifted to the non-OSI-approved Elastic Licence 2.0 in 2021 and gates critical features (end-point response, ML anomaly detection, CSPM) behind Platinum/Enterprise subscriptions (\$96–\$175/node/month). TheHive 5.x introduced a Community/Enterprise split limiting multi-tenancy and MISP integration.
2. **XDR-level cross-signal correlation.** Wazuh’s hierarchical rule engine natively correlates events across host (FIM, rootkit, process), network (Suricata alerts, Zeek metadata), cloud (CloudTrail, GuardDuty), and Kubernetes (audit logs, container runtime) domains. This cross-domain correlation is the definition of XDR. Security Onion excels at network monitoring but lacks a host/cloud correlation engine. OSSIM’s correlation engine is limited and unmaintained since 2020. TheHive+Cortex is not a detection tool at all.
3. **Sub-30-second automated response.** Wazuh’s active response framework executes remediation on the agent side within seconds — the manager sends a response command and the agent executes locally (firewall-drop, host-deny, custom scripts). This architecture eliminates round-trip latency. The project extends this with a pull-based Response Agent where spoke-cluster controllers poll Wazuh for CiliumNetworkPolicy isolation actions. No other platform achieves this without commercial licensing.
4. **Native Kubernetes agent.** Wazuh Agent runs as a DaemonSet on every EKS node, collecting container events, monitoring file integrity, running CIS benchmarks, and decoding K8s audit logs. OSSIM and TheHive have no Kubernetes awareness. Security Onion’s osquery is query-driven (pull), not real-time.
5. **Suricata/ZeeK/Falco integration.** Wazuh natively ingests Suricata `eve.json` via dedicated module, Zeek logs via Fluent Bit, and Falco alerts via HTTP output — all available for cross-signal correlation in the rule engine.
6. **Bare-metal portability.** The entire stack (Manager, Indexer, Dashboard, Agents) runs on bare metal with zero cloud dependencies. Identical configuration between cloud and on-premises deployments.

Note on TheHive + Cortex: This stack is complementary, not competing. It could be added as a Tier 2/3 investigation layer on top of Wazuh for enhanced case management in the future. MISP is already integrated independently for threat intelligence.

5 Container Runtime Security (Syscall Monitoring)

Selected: Falco

5.1 Candidates Evaluated

Criterion	Falco	Tracee	KubeArmor
Primary Function	Detection (IDS)	Detection + Forensics	Enforcement (IPS)
Data Source	Syscall tracepoints	Syscalls + kprobes + LSM hooks + network	LSM hooks + eBPF telemetry
Driver	Kernel module or eBPF (CO-RE)	eBPF only (CO-RE)	LSM (AppArmor/BPF-LSM/SELinux) + eBPF
Min Kernel Version	4.14 (eBPF), 3.x (kmod)	5.4 (BTF required)	4.15 (AppArmor), 5.7 (BPF-LSM)
Rule Language	Falco Rules YAML DSL	Rego (OPA) or Go signatures	KubeArmorPolicy CRD (YAML)
Prebuilt Rules	100+ (MITRE ATT&CK mapped)	CVE-specific + behavioural	Policy templates
Inline Enforcement	No (detection-only by design)	Limited (LSM hooks, newer)	Yes (primary feature)
K8s Audit Logs	Yes (plugin)	No	No
Output Routing	falcosidekick (60+ destinations)	JSON/gRPC	Telemetry (JSON)
CNCF Status	Graduated (highest maturity)	Sandbox	Sandbox
GitHub Stars	~7,500	~3,800	~1,500
CPU Overhead	1–3% per node	1–4% per node	<1% per node
Memory	50–150 MB	80–200 MB	30–80 MB
ARM64	Full support	Full support	Full support
Licence	Apache 2.0	Apache 2.0	Apache 2.0
Commercial Backer	Sysdig	Aqua Security	AccuKnox
Bare-Metal	Yes (standalone binary)	Yes (standalone binary)	Yes (daemon mode)

5.2 Selection Rationale

Falco was selected for container runtime threat detection for four reasons:

1. **CNCF Graduated maturity.** Falco is the only CNCF Graduated runtime security tool (alongside Kubernetes, Prometheus, Envoy). This signals battle-tested production readiness, broad enterprise adoption, and long-term community sustainability. Tracee and KubeArmor are both CNCF Sandbox projects with smaller communities.
2. **Broad detection rule ecosystem.** Falco’s 100+ prebuilt rules cover the MITRE ATT&CK framework: container escape, shell-in-container, privilege escalation, sensitive file access, unexpected network connections, package manager execution in production containers. The community rule library is the largest of any container security tool.
3. **Integration with Wazuh XDR.** falcosidekick routes Falco alerts to Wazuh via HTTP output, enabling cross-signal correlation: a Falco alert (container escape attempt) + Suricata alert (C2 beacon) + Wazuh FIM alert (binary modification) correlate into a single high-confidence composite alert.

4. **Complementary role with Tetragon.** Falco detects and alerts; Tetragon detects and enforces (see Section 6). Falco provides the broadest detection coverage with the most community rules; Tetragon provides in-kernel kill/block enforcement. This detection + enforcement separation follows defence-in-depth principles.

Why not Tracee? Tracee provides deeper kernel visibility (kprobes, 400+ events) and unique forensic capabilities, but overlaps heavily with both Falco (detection) and Tetragon (eBPF tracing). Its Rego-based signature language adds operational complexity. The project already achieves forensic coverage via Zeek + Wazuh.

Why not KubeArmor? KubeArmor’s LSM-based enforcement is complementary to Tetragon’s eBPF enforcement but adds operational complexity. Its AppArmor dependency can be limiting on Amazon Linux 2 (which defaults to neither AppArmor nor SELinux). Could be considered as a future addition for strict allowlist enforcement on specific high-security research pods.

6 eBPF Runtime Security (Process/File/Network Enforcement)

Selected: Tetragon

6.1 Candidates Evaluated

Criterion	Tetragon	Tracee (eBPF)	bpfttrace	Inspektor Gadget
Primary Function	Detection + Enforcement	Detection + Forensics	Ad-hoc tracing/investigation	Debugging + observability
Production Use	Yes (always-on DaemonSet)	Yes (always-on DaemonSet)	No (temporary sessions)	Partial (debugging tool)
Hook Types	kprobes, tracepoints, LSM, uprobes	kprobes, tracepoints, LSM, uprobes, TC, XDP	kprobes, tracepoints, uprobes, USDT, PMCs	kprobes, tracepoints (gadget-based)
Inline Enforcement	Yes (SIGKILL, Override, LSM block)	Limited (SIGKILL, LSM — newer)	No	No
In-Kernel Kill	Yes (sub-millisecond, no userspace round-trip)	Yes (SIGKILL)	No	No
Policy Language	TracingPolicy CRD (YAML)	Rego (OPA) / Go signatures	bpfttrace scripting language	Gadget parameters
Process Tree	In-kernel eBPF maps (zero userspace cost)	In-kernel eBPF maps	Manual (per-script)	N/A
K8s Awareness	Native (Cilium integration, pod metadata)	Yes (metadata enrichment)	None	Yes (kubectrl plugin)
Cilium Integration	Native (shared eBPF maps, Hubble visibility)	No	No	Partial
CNCF Status	Part of Cilium ecosystem (Graduated)	Sandbox	N/A (kernel tool)	Sandbox
CPU Overhead	1–2%	1–4%	Variable (0–15%)	Variable
Licence	Apache 2.0	Apache 2.0	Apache 2.0	Apache 2.0

6.2 Selection Rationale

Tetragon was selected for eBPF-based runtime enforcement for three reasons:

1. **In-kernel enforcement without userspace round-trip.** Tetragon can SIGKILL a process or override a kernel function return value (returning `-EPERM` to block an operation) entirely within the eBPF program — sub-millisecond response with zero userspace latency. This is critical for the <30-second detection-to-block XDR requirement. Falco cannot enforce at all; Tracee’s enforcement is newer and less mature.
2. **Native Cilium integration.** Tetragon shares eBPF maps with Cilium for pod identity and metadata resolution, and its events are visible in Hubble UI alongside Cilium network flows. This creates a unified eBPF-based security plane across network (Cilium), runtime (Tetragon), and observability (Hubble).
3. **TracingPolicy CRD flexibility.** Tetragon’s TracingPolicy CRD allows hooking any kernel function, not just syscall boundaries. This provides more precise detection: hooking `security_file_open` (LSM) instead of the generic `open(2)` syscall reduces noise and overhead. Custom policies can enforce binary hash verification, process credential changes, and kernel module loading.

bpfftrace role in the architecture: bpfftrace is retained as an incident response tool available on bastion hosts and via privileged debug pods, not as a permanent DaemonSet. Its ad-hoc tracing capabilities are invaluable for security investigation.

7 CNI / Software-Defined Networking

Selected: Cilium + Hubble

7.1 Candidates Evaluated

Criterion	Cilium + Hubble	Calico	Weave Net	Flannel
Datapath	eBPF (native, purpose-built)	iptables (default) or eBPF (added later)	iptables + OVS	VXLAN / host-gw / WireGuard
K8s NetworkPolicy v1	Full support	Full support	Full support	None
Custom Policy CRDs	CiliumNetworkPolicy, CiliumClusterwideNetworkPolicy	CalicoNetworkPolicy, GlobalNetworkPolicy	None	None
L3/L4 Policy	Yes (eBPF, O(1) lookups)	Yes (iptables O(n) or eBPF)	Yes (iptables)	No
L7 Policy (HTTP/-gRPC/DNS/Kafka)	Yes (native, free)	Enterprise only (paid)	No	No
DNS-Aware FQDN Policies	Yes (inline DNS interception, dynamic IP resolution)	Limited (periodic DNS resolution)	No	No
Identity-Based Policies	Yes (Cilium Identity, label-derived)	Partial (label selectors)	No	No
kube-proxy Replacement	Yes (eBPF socket-based LB)	Yes (eBPF mode)	No	No
WireGuard Encryption	Yes (kernel-native, single toggle)	Yes	No (NaCl userspace, slow)	Yes (as backend)
Flow Observability	Hubble (L3–L7 flows, CLI, UI, Prometheus)	Enterprise only	Minimal	None
DNS Logging	Yes (Hubble DNS)	Enterprise only	No	No
Service Map UI	Hubble UI (free)	Enterprise only	No	No
Host Firewall	Yes (CiliumClusterwideNetworkPolicy)	Yes (HostEndpoint)	No	No
BGP Support	Yes (Cilium BGP Control Plane)	Yes (mature, native)	No	No
CNCF Status	Graduated	N/A (Tigera commercial)	N/A (Weaveworks defunct)	N/A
GitHub Stars	~20,500	~6,000	~6,600	~8,800
Active Development	Very active	Active	Maintenance mode (company shut down 2024)	Minimal
Performance at Scale	Excellent (O(1) eBPF hash map lookups)	Good (eBPF) / Moderate (iptables O(n))	Moderate	Good (simple forwarding)
Licence	Apache 2.0	Apache 2.0 (OSS) / Commercial (Enterprise)	Apache 2.0	Apache 2.0
Bare-Metal	Yes (VXLAN, Geneve, native routing, BGP)	Yes (native, BGP, VXLAN, IP/IP)	Yes (VXLAN)	Yes (VXLAN, host-gw)

7.2 Selection Rationale

Cilium was selected as the CNI/SDN for five decisive advantages:

1. **L7 network policy in open-source.** Cilium is the only CNI providing native L7 policy

enforcement (HTTP method/path, gRPC service/method, DNS FQDN, Kafka topic) in its free Apache 2.0 edition. Calico gates L7 policy behind its commercial Enterprise product. For zero-trust architecture, L7 visibility and enforcement are essential — allowing traffic to port 443 is insufficient; policies must specify which HTTPS endpoints are permitted.

2. **DNS-aware FQDN allowlisting.** Cilium intercepts DNS queries and dynamically resolves FQDNs to IPs, creating L3 rules matching current addresses. This enables policies like “allow egress to api.github.com” without hardcoding IPs — critical for research workloads that access external APIs.
3. **WireGuard transparent encryption.** A single Helm value enables kernel-native WireGuard encryption for all pod-to-pod traffic between nodes. This provides intra-cluster encryption for the zero-trust model without per-service TLS configuration.
4. **Hubble flow observability.** Hubble provides L3–L7 flow logs with policy verdict annotation, DNS query logging, HTTP request/response metadata, and a service map UI — all free. This feeds directly into Grafana dashboards for security monitoring. Calico’s equivalent flow visualisation requires Enterprise licensing.
5. **CNCF Graduated + eBPF-first architecture.** Cilium is CNCF Graduated (highest maturity tier), designed from the ground up around eBPF. Its datapath provides $O(1)$ policy evaluation via hash maps, replacing iptables’ $O(n)$ linear rule scanning. This performance advantage is critical at scale.

Why not Calico? Calico’s open-source edition is a capable CNI with mature BGP support, but its most valuable security features (L7 policy, flow visualisation, DNS logging, compliance) are gated behind the commercial Enterprise product. Calico’s policy tier system (ordered evaluation) is unique and valuable, but insufficient to offset the L7 and observability gaps.

Why not Flannel? Flannel provides only pod networking with zero network policy support. It requires a separate policy engine (Calico in “Canal” mode) for any security enforcement. Not viable for a security-focused architecture.

Why not Weave Net? Weaveworks shut down in February 2024. The project is in maintenance mode with no backing company. Not recommended for new deployments.

8 Kubernetes Admission Control

Selected: Kyverno

8.1 Candidates Evaluated

Criterion	Kyverno	OPA/Gatekeeper	Kubewarden	jsPolicy
Policy Language	YAML (K8s native)	Rego (domain-specific)	Any (Wasm-compiled)	JavaScript/TypeScript
Learning Curve	Low (YAML)	High (Rego DSL)	Medium (Wasm toolchain)	Low (JS/TS)
Validation	Yes	Yes	Yes	Yes
Mutation	Yes (mature)	Yes (later addition, less mature)	Yes	Yes
Image Verification	Yes (Cosign, Notary — built-in)	Via external tools	Yes (Sigstore)	No
Resource Generation	Yes (unique: auto-generate NetworkPolicies, ConfigMaps)	No	No	Controller mode
Background Scan	Yes (scan existing resources for compliance)	Yes (audit)	Yes (Audit Scanner)	Controller mode
Policy Reports	Yes (PolicyReport CRD — standardised)	Via audit controller	Yes	No
CNCF Status	Incubating	Graduated	Sandbox	None
GitHub Stars	~5,500	OPA: ~9,500 / Gatekeeper: ~3,700	~1,000	~500
Pre-built Policies	Extensive library (Pod Security Standards, best practices)	Extensive (gatekeeper-library)	Growing	Limited
Licence	Apache 2.0	Apache 2.0	Apache 2.0	Apache 2.0
Bare-Metal	Excellent	Excellent	Excellent	Excellent

8.2 Selection Rationale

Kyverno was selected for three reasons:

- Kubernetes-native policies (no new language).** Kyverno policies are written in standard YAML as Kubernetes resources. This eliminates the need to learn Rego (OPA’s domain-specific language), reducing the operational burden on the team. Security policies should be readable by anyone who understands Kubernetes manifests.
- Built-in image verification.** Kyverno natively verifies container image signatures (Cosign/Sigstore) and attestations as part of admission control. This enables policies like “only admit pods with images signed by the CI pipeline” — a critical supply chain security control. OPA/Gatekeeper requires external tools for image verification.
- Mutation + Validation + Generation in one tool.** Kyverno uniquely combines all three admission control patterns: validate (reject non-compliant resources), mutate (auto-add security labels, resource limits), and generate (auto-create NetworkPolicies for new namespaces). OPA/Gatekeeper added mutation support later but it remains less mature.

Why not OPA/Gatekeeper? OPA is CNCF Graduated (higher maturity tier than Kyverno’s Incubating) and Rego is more powerful for complex policy logic. However, Rego’s learning curve is a significant operational cost. OPA/Gatekeeper would be the correct choice if the team had existing OPA expertise or if policies require complex logic beyond YAML’s expressiveness. For this project’s policy requirements (registry allowlist, PSS enforcement, resource limits, image signing), Kyverno’s YAML-based approach is sufficient and more maintainable.

9 Threat Intelligence Platform

Selected: MISP

9.1 Candidates Evaluated

Criterion	MISP	OpenCTI	CRITs	Yeti
Data Model	Events + Attributes (IOCs)	STIX 2.1 Knowledge Graph	Objects + Relationships	Observables + Indicators
STIX Support	STIX 1.x + 2.x (import/export)	STIX 2.1 native	STIX 1.x only	STIX export
ATT&CK Integration	Galaxy clusters (comprehensive)	Native TTPs (excellent)	Limited	Manual
Sharing Communities	Excellent (CIRCL, NATO, ISACs, global CERTs)	Good (connectors)	Basic	Limited
Feed Ingestion	Extensive (OSINT + commercial feeds, automated)	60+ connectors (automated)	Limited	Configurable
Suricata/Zeek Integration	Direct (MISP → suricata-update, Zeek Intel framework)	Via connectors (indirect)	No	No
API	Full REST API + PyMISP library	GraphQL API + Python client	Basic REST	REST API
K8s Deployment	Docker images, community Helm charts	Docker Compose, community Helm	No K8s support	Docker images
Backend	MySQL/MariaDB + Redis	Elasticsearch + Redis + RabbitMQ/Kafka + MinIO	MongoDB	MongoDB + Redis
Resource Requirements	Moderate (4 vCPU, 8 GB RAM)	Heavy (8+ GB RAM, multiple services)	Moderate	Light
Licence	AGPLv3	Apache 2.0 (core) / Commercial (Enterprise)	MIT	Apache 2.0
Community	Very large (CIRCL, global CERTs, ~5K+ stars)	Large and growing (Filigran, ~4K+ stars)	Stagnant (MITRE deprioritised)	Small
Maintenance	Very active	Very active	Effectively unmaintained	Active (small team)
Bare-Metal	Good (packages, automated installer)	Moderate (Docker preferred, many dependencies)	Moderate	Good

9.2 Selection Rationale

MISP was selected as the threat intelligence platform for three reasons:

1. **Direct integration with detection tools.** MISP feeds IOCs directly into Suricata (via `suricata-update` MISP feed support) and Zeek (via the Intel framework). This creates an automated threat intelligence → detection pipeline: new IOCs added to MISP are immediately available as Suricata signature updates and Zeek intel matches, without manual rule writing.
2. **Industry-standard sharing platform.** MISP is the de facto standard for threat intelligence sharing, used by NATO, FIRST community, national CERTs, and ISACs worldwide. Its sharing groups, synchronisation protocol, and STIX/TAXII support enable participation in threat intelligence communities relevant to research infrastructure protection.
3. **Maturity and community scale.** MISP has been in active development since 2012, maintained by CIRCL (Computer Incident Response Center Luxembourg). Its Galaxy system provides comprehensive MITRE ATT&CK mapping for all threat intelligence objects.

Why not OpenCTI? OpenCTI's STIX 2.1 native knowledge graph and modern architecture are technically superior. However, its resource requirements are significantly higher (Elasticsearch + Redis + RabbitMQ + MinIO minimum), and its direct integration with Suricata/Zeek is less mature than MISP's. OpenCTI could be added alongside MISP (with bidirectional sync) for enhanced threat analysis and visualisation in the future.

Why not CRITs? CRITs development has effectively ceased since ~2020. Not recommended for new deployments.

10 Log Shipping / Collection

Selected: Fluent Bit

10.1 Candidates Evaluated

Criterion	Fluent Bit	Fluentd	Vector	Logstash	Filebeat
Language	C	Ruby + C	Rust	Java (JVM)	Go
Memory Usage	~5–15 MB	~40–100 MB+	~10–30 MB	~200–500 MB	~20–50 MB
CPU Overhead	Very low	Moderate	Very low	High	Low
Transform Power	Medium (Lua scripting)	High (Ruby plugins)	High (VRL — type-safe)	High (Grok, Ruby)	Low (processors)
Plugin Ecosystem	Medium (growing, covers most use cases)	Very large (1,000+ plugins)	Medium	Large (200+ plugins)	Medium (modules)
K8s Metadata	Native filter (pod, namespace, labels)	Plugin	Native source	Plugin	Autodiscovery
Multi-Signal	Logs + metrics	Logs primarily	Logs + metrics + traces	Logs primarily	Logs primarily
Dual Output	Yes (Wazuh + S3 simultaneously)	Yes	Yes	Yes	Yes
CNCF Status	Graduated (under Fluentd project)	Graduated	No	No	No
Licence	Apache 2.0	Apache 2.0	MPL 2.0	Elastic Licence 2.0 (NOT OSI)	Elastic Licence 2.0 (NOT OSI)
Bare-Metal	Excellent (single binary)	Good (Ruby dependency)	Excellent (single binary)	Good (JVM dependency)	Excellent (single binary)

10.2 Selection Rationale

Fluent Bit was selected for three reasons:

1. **Minimal resource footprint.** At ~5–15 MB memory per node, Fluent Bit is an order of magnitude lighter than Logstash (~200–500 MB) and significantly lighter than Fluentd (~40–100 MB). On EKS nodes running Wazuh Agent, Falco, Tetragon, and Cilium alongside application workloads, every megabyte of DaemonSet overhead matters.
2. **CNCF Graduated status + Apache 2.0 licence.** Fluent Bit (as part of the Fluentd project) is CNCF Graduated, ensuring long-term community sustainability. Its Apache 2.0 licence is fully open-source. Filebeat and Logstash are under Elastic Licence 2.0 (not OSI-approved), disqualifying them under the project’s licensing constraint.
3. **Dual-output architecture.** Fluent Bit natively supports multiple outputs from a single pipeline, enabling the project’s dual-shipping pattern: security logs → Wazuh Manager for real-time correlation AND security logs → S3 (Object Lock) for immutable forensic preservation. This is critical for evidence chain integrity.

Why not Vector? Vector (Rust) is technically excellent with superior transform capabilities (VRL language) and comparable resource efficiency. However, its MPL 2.0 licence (weak copyleft) and Datadog ownership introduce concerns. Fluent Bit’s CNCF backing and wider Kubernetes ecosystem adoption make it the safer choice.

11 Monitoring Stack (Metrics + Visualisation + Logs)

Selected: Prometheus + Grafana + Loki

11.1 Candidates Evaluated

Criterion	PLG (Prometheus + Grafana + Loki)	ELK (Elasticsearch + Logstash + Kibana)	VictoriaMetrics + Grafana	Graylog
Primary Focus	Metrics + Logs + Visualisation	Full-text log search + Analytics	Metrics (+ Loki for logs)	Centralised log management
Metrics Collection	Prometheus (pull-based, PromQL)	Metricbeat (push-based)	VictoriaMetrics (PromQL-compatible)	Limited
Log Aggregation	Loki (label-based indexing, cheap storage)	Elasticsearch (full-text indexing, expensive storage)	Loki (same)	Elasticsearch-backed
Visualisation	Grafana (best-in-class dashboards)	Kibana (good, Elastic-specific)	Grafana (same)	Graylog UI
Query Language	PromQL (metrics) + LogQL (logs)	KQL / Lucene / ES QL	MetricsQL (PromQL superset) + LogQL	Graylog QL
Full-Text Search	No (Loki indexes labels, not content)	Yes (Lucene-based, industry-leading)	No (metrics only)	Yes (Elasticsearch-backed)
K8s Integration	Excellent (kube-prometheus-stack, ServiceMonitor CRDs)	Good (ECK operator)	Good (VM operator, ServiceMonitor compatible)	Moderate
Resource Usage	Low–Medium	High (Elasticsearch is memory-hungry, JVM)	Very Low (7–10x less storage than Prometheus)	Medium–High
Storage Cost	Low (Loki: label-indexed, S3 backend)	High (full-text index, SSD-heavy)	Very Low (70x compression)	Medium
Licence	Apache 2.0 (Prometheus) / AGPLv3 (Grafana, Loki)	Elastic Licence 2.0 (NOT OSI)	Apache 2.0 (OSS) / Commercial (Enterprise)	SSPL (NOT OSI) since v5.0
CNCF	Prometheus: Graduated	No	No	No
Bare-Metal	Excellent (Go binaries, no cloud deps)	Good (JVM, heavier)	Excellent (Go binary)	Good (JVM + Elasticsearch)

11.2 Selection Rationale

The **PLG stack** was selected for four reasons:

1. **Cloud-native standard.** Prometheus is the CNCF Graduated monitoring standard for Kubernetes. The kube-prometheus-stack Helm chart provides a complete, opinionated deployment with ServiceMonitor/PodMonitor CRDs for automatic metric scraping. Every Kubernetes tool in the architecture (Cilium, Falco, Wazuh, Tetragon) exports Prometheus metrics natively.
2. **Cost-effective log aggregation.** Loki indexes only labels (pod name, namespace, severity), not log content, making it dramatically cheaper to operate than Elasticsearch at the same log volume. For security log archival where most queries filter by time range and source, label-based indexing is sufficient. S3-backed storage further reduces costs.
3. **Grafana visualisation.** Grafana provides the best-in-class dashboard experience with native data sources for Prometheus (metrics), Loki (logs), and the Wazuh Indexer (OpenSearch — for security event investigation). A single visualisation platform for both operational monitoring and security dashboards.
4. **Licensing.** Prometheus is Apache 2.0. Grafana and Loki are AGPLv3, which permits internal use without restriction (AGPL's source-disclosure requirement only applies when providing the software as a network service to third parties). ELK (Elastic Licence 2.0) and Graylog (SSPL since v5.0) use non-OSI-approved licences that conflict with the project's open-source mandate.

Note on VictoriaMetrics: VictoriaMetrics is an excellent Prometheus-compatible time-series database with superior compression (7–10x less storage) and a PromQL superset (Metric-sQL). It is recommended as a future Prometheus replacement if long-term metric retention or cost optimisation becomes important — it is a drop-in replacement requiring no dashboard or alerting rule changes.

12 Web Application Firewall (WAF)

Selected: ModSecurity / Coraza (sidecar with cloudflared)

12.1 Candidates Evaluated

Criterion	ModSecurity 3.x	Coraza WAF	lua-resty-waf
Language	C/C++ (libmodsecurity)	Go	Lua (OpenResty)
CRS Compatibility	Full (reference implementation)	High (most CRS v4 rules)	Partial (translated)
Web Server Support	NGINX, Apache, IIS	Caddy, NGINX, Envoy (Wasm)	OpenResty only
Envoy/Istio	No	Yes (coraza-proxy-wasm)	No
K8s Integration	NGINX Ingress (built-in option)	Envoy/Istio/Caddy (cloud-native)	OpenResty only
Performance	Medium (regex overhead)	Good (Go native)	Good (LuaJIT)
Maintenance	Uncertain (Trustwave EOL 2024, community takeover)	Active (OWASP project)	Unmaintained
Licence	Apache 2.0	Apache 2.0	MIT
Community	Large (legacy, declining)	Growing (positioned as ModSecurity successor)	Small
Bare-Metal	Excellent	Excellent	Moderate

12.2 Selection Rationale

The architecture deploys WAF as a **sidecar alongside cloudflared pods**, inspecting L7 HTTP traffic before it reaches application services. Both ModSecurity and Coraza are viable:

- **ModSecurity** is the industry standard with the deepest CRS compatibility, but Trustwave announced end-of-life for commercial support in 2024, creating maintenance uncertainty.
- **Coraza** is the modern Go-based successor, OWASP-backed, with CRS v4 compatibility and native Envoy/Wasm integration for cloud-native deployment.

Recommendation: Use ModSecurity for existing NGINX-based deployments; adopt Coraza for new deployments and plan migration. Both use OWASP Core Rule Set v4 for protection against OWASP Top 10 threats (SQLi, XSS, RCE, LFI/RFI).

13 NAT Gateway (Software-Based)

Selected: iptables/nftables NAT VM

13.1 Candidates Evaluated

Criterion	iptables/nftables NAT VM	AWS NAT Gateway	pfSense/OPNsense	VyOS
Type	Self-managed Linux VM	Fully managed AWS service	Self-managed firewall appliance	Self-managed network OS
NAT	SNAT/DNAT (full control)	SNAT only	SNAT/DNAT + firewall	SNAT/DNAT + routing
Firewall / Filtering	Full (iptables/nftables rules)	None (NAT only)	Full (pf-based)	Full (nftables/iptables)
PCAP Capability	Yes (tcpdump on NAT VM)	No	Yes	Yes
Connection Tracking	Full (conntrack, tunable)	Automatic (350s idle timeout)	Full	Full
Performance	Medium–Good (nftables O(1) sets)	Excellent (100 Gbps, auto-scaling)	Good (FreeBSD pf)	Good
HA	Manual (VRRP/keepalived, HA pair)	Automatic (per-AZ)	CARP HA pair	VRRP HA pair
Monthly Cost	~\$16–60 (t3.medium HA pair)	~\$77–232+ (hourly + \$0.045/GB)	~\$30–60 (VM cost)	~\$30–60 (VM cost)
Bare-Metal	Yes (Linux kernel built-in)	No (AWS-only)	Yes (FreeBSD-based)	Yes (Debian-based)
Customisation	Full (any iptables/nftables rule)	None	Full (pf rules + GUI)	Full (CLI + config)
Cloud Lock-in	None	Complete (AWS-specific service)	None	None
Licence	GPLv2 (Linux kernel)	N/A (managed service)	Apache 2.0 (pfSense CE) / BSD (OPNsense)	GPLv2

13.2 Selection Rationale

iptables/nftables NAT VMs were selected for three reasons:

1. **Bare-metal portability.** AWS NAT Gateway is a proprietary managed service with no equivalent on bare metal. Since the architecture must be replicable on-premises, the NAT function must use standard Linux networking. iptables/nftables are built into the Linux kernel and work identically on AWS EC2, bare-metal servers, and any Linux VM.
2. **Firewall + NAT + PCAP in one.** The NAT VM provides not just SNAT for private subnet egress, but also stateful packet filtering (complementing Suricata IPS), connection tracking visibility, and tcpdump capability for network forensics. AWS NAT Gateway provides NAT only — no filtering, no PCAP, no customisation.
3. **Cost efficiency.** A t3.medium HA pair costs ~\$60/month versus AWS NAT Gateway at ~\$77–232+/month (per-AZ hourly charges + \$0.045/GB data processing). For research workloads with variable egress patterns, the NAT VM approach avoids per-GB data processing charges.

nftables vs iptables: The architecture uses iptables for compatibility but should migrate to nftables for new deployments. nftables provides $O(1)$ set-based lookups (vs iptables' $O(n)$ linear rule evaluation), atomic rule updates, unified IPv4/IPv6 handling, and a cleaner syntax. All modern Linux distributions default to nftables.

Why not pfSense/OPNsense? These are excellent firewall appliances but add operational complexity (FreeBSD-based OS, separate management UI) beyond what's needed for a NAT gateway function. The architecture already has Suricata for deep packet inspection; the NAT VM's firewall role is supplementary.

14 Licensing Summary

Table 3: Licensing summary for all selected tools

Tool	Licence	OSI?	Feature Gating?	Notes
Suricata 7.x	GPLv2	Yes	None	OISF foundation
Zeek 7.x	BSD	Yes	None	Most permissive
Wazuh 4.x	GPLv2 + Apache 2.0	Yes	None	All features free
Falco	Apache 2.0	Yes	None	CNCF Graduated
Tetragon	Apache 2.0	Yes	None	Part of Cilium
Cilium + Hubble	Apache 2.0	Yes	None	CNCF Graduated
Kyverno	Apache 2.0	Yes	None	CNCF Incubating
MISP	AGPLv3	Yes	None	Internal use unrestricted
Fluent Bit	Apache 2.0	Yes	None	CNCF Graduated
Prometheus	Apache 2.0	Yes	None	CNCF Graduated
Grafana	AGPLv3	Yes	None	Internal use unrestricted
Loki	AGPLv3	Yes	None	Internal use unrestricted
ModSecurity/Coraza	Apache 2.0	Yes	None	OWASP project
iptables/nftables	GPLv2	Yes	None	Linux kernel

All selected tools use OSI-approved open-source licences with zero feature gating. AGPLv3 (MISP, Grafana, Loki) requires source disclosure only when providing the software as a network service to third parties — internal use within Big Chemistry infrastructure is unrestricted.

Table 4: Disqualified alternatives due to licensing

Tool	Licence	Disqualification Reason
Elastic Security	Elastic Licence v2 (ELv2)	Not OSI-approved; critical features (endpoint response, ML, CSPM) require Platinum subscription
Filebeat	Elastic Licence v2	Not OSI-approved
Logstash	Elastic Licence v2	Not OSI-approved
Calico Enterprise	Commercial (Tigera)	L7 policy, flow visualisation, DNS logging, compliance require paid licence
Graylog 5.0+	SSPL	Not OSI-approved since v5.0
AWS NAT Gateway	N/A (proprietary service)	No bare-metal equivalent

14.1 Disqualified Alternatives (Licensing)

15 Architecture Integration Map

15.1 Current Deployment: Monitoring-Only XDR Overlay

The **current deployment** is a monitoring-only XDR overlay on the existing FourCo infrastructure. No infrastructure components were changed; all security tools were added on top.

The architecture operates across five layers:

Threat Intelligence

MISP runs on a dedicated EC2 instance, distributing IOCs to Zeek (via the Intel framework) and Wazuh (via CDB lists) for real-time matching against observed network and host activity.

Detection & Correlation

Zeek 7.x performs passive network security monitoring via VPC Traffic Mirroring, generating structured protocol logs. Wazuh Manager 4.x serves as the central XDR/SIEM, correlating events from all sources through its hierarchical rule engine and issuing active response commands.

Node-Level Security (DaemonSets)

Every EKS node in both VPCs runs three DaemonSets: Falco (container runtime IDS, detection-only), Tetragon (eBPF tracing, observability-only), and Wazuh Agent 4.x (host IDS providing FIM, rootkit detection, vulnerability scanning, and CIS benchmarking).

Log Shipping & Response

Fluent Bit collects logs from all security tools and ships them to two destinations simultaneously: Wazuh Manager for real-time correlation and S3 with Object Lock for immutable forensic preservation. Automated response is handled by a custom controller and AWS Lambda functions that modify Security Groups, disable IAM keys, and update WAF rules.

Monitoring & Existing Services

Prometheus scrapes metrics from all security tools, visualised alongside Loki logs in Grafana dashboards. AWS-native security services (GuardDuty, CloudTrail, VPC Flow Logs, Inspector, ALB/WAF logs) feed findings into Wazuh Manager. The existing FourCo infrastructure (Route53, AWS WAF, ALB, EKS with vpc-cni, AWS NAT Gateways, bastion hosts, VPC peering) remains unchanged.

15.2 Recommended Target: Full Security Architecture

The **recommended target state** extends the monitoring overlay into a full inline security architecture with enforcement capabilities. The key architectural additions are:

Perimeter Security

Suricata IPS deployed in an inline sandwich configuration (ingress + egress inspection), with rules fed from MISP via `suricata-update`. EVE JSON logs flow through Fluent Bit to Wazuh for correlation.

Network Layer

Cilium replaces vpc-cni as the CNI, providing eBPF-based L3–L7 network policy enforcement, DNS-aware FQDN allowlisting, WireGuard transparent encryption, and Hubble flow observability. Software-based NAT VMs (iptables/nftables) replace AWS NAT Gateways for bare-metal portability, adding stateful filtering and PCAP capability.

Runtime Enforcement

Tetragon transitions from observability-only to active enforcement mode, providing in-kernel process kill and operation blocking via TracingPolicy CRDs. Falco continues as the detection layer with its broader community rule set.

Admission Control

Kyverno enforces supply chain security (container image signature verification), Pod Security Standards, and resource limits at the Kubernetes API level.

Application Security

ModSecurity/Coraza WAF deploys as a sidecar alongside cloudflared pods, inspecting L7 HTTP traffic with OWASP CRS v4 rules before it reaches application services.

Response

A pull-based Response Agent enables Wazuh to trigger CiliumNetworkPolicy-based pod isolation, complementing the existing Lambda-based AWS-level responses (Security Group modification, IAM key disabling, WAF rule updates).

15.3 Key Differences Between Current and Target

Table 5: Key differences between current deployment and recommended target

Aspect	Current (15.1)	Target (15.2)
Perimeter	No inline IPS	Suricata IPS sandwich (inline blocking)
CNI	vpc-cni (AWS native)	Cilium (eBPF, L7 policy, WireGuard)
Runtime	Tetragon observe-only, Falco detect-only	Tetragon enforce, Falco detect
Admission	None	Kyverno (image signing, PSS, limits)
App WAF	AWS WAF only	AWS WAF + Coraza CRS v4 sidecar
NAT	AWS NAT Gateway (managed)	NAT VM (iptables/nftables, portable)
Response	Lambda (SG/IAM/WAF)	Cilium NP isolation + Lambda

This document provides the technical justification for every security tool selected in the Big Chemistry XDR architecture. All tools are fully open-source (OSI-approved licences), portable to bare-metal infrastructure, and actively maintained by substantial communities.